

Servlets, JDBC, Sessions, MVC Refactor & Spring — Assignment Answers

Topics: hardcoded servlet output, JDBC insert, HttpSession, MVC refactor, Spring MVC comparison

Q1. PlaylistServlet — Hardcoded Favorite Playlists

```
@WebServlet("/playlists")
public class PlaylistServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        String[] playlists = {
            "Late Night Vibes",
            "Workout Hits 2026",
            "Bollywood Retro Gold"
        };

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<h2>My Favorite Spotify Playlists</h2><ul>");
        for (String playlist : playlists) {
            out.println("<li>" + playlist + "</li>");
        }
        out.println("</ul>");
    }
}
```

Visiting /playlists lists the three hardcoded playlists as bullet points.

Q2. AddRestaurant.jsp — Storing Restaurant Details via JDBC

AddRestaurant.jsp form:

```
<form action="AddRestaurantServlet" method="POST">
  <input type="text" name="restaurantName" placeholder="Restaurant name" required>
  <select name="cuisine" required>
    <option value="North Indian">North Indian</option>
    <option value="Chinese">Chinese</option>
    <option value="Italian">Italian</option>
  </select>
  <button type="submit">Add Restaurant</button>
</form>
```

AddRestaurantServlet.java (JDBC INSERT):

```
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

    String name = request.getParameter("restaurantName");
    String cuisine = request.getParameter("cuisine");

    String sql = "INSERT INTO restaurants (name, cuisine) VALUES (?, ?)";

    try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER, DB_PASS);
        PreparedStatement stmt = conn.prepareStatement(sql)) {

        stmt.setString(1, name);
        stmt.setString(2, cuisine);
        stmt.executeUpdate();

        response.getWriter().println("<h2>Restaurant '" + name + "' (" + cuisine + ") added successfully!</h2>");

    } catch (SQLException e) {
        e.printStackTrace();
        response.getWriter().println("<h2>Sorry, we couldn't save the restaurant right now.</h2>");
    }
}
```

Q3. Session Tracking — LoginServlet + HomeServlet

```
// LoginServlet.java
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

    String username = request.getParameter("username");
    String password = request.getParameter("password");

    if (VALID_USER.equals(username) && VALID_PASS.equals(password)) {
        HttpSession session = request.getSession();
        session.setAttribute("username", username);
        response.sendRedirect("HomeServlet");
    } else {
        response.getWriter().println("<h3>Invalid username or password</h3>");
    }
}

// HomeServlet.java
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

    HttpSession session = request.getSession(false);
    String username = (session != null) ? (String) session.getAttribute("username") : null;

    if (username == null) {
        response.sendRedirect("login.html");
        return;
    }

    response.getWriter().println("<h2>Welcome back, " + username + "!</h2>");
}
```

Behaviour: logging in via LoginServlet stores the username in the session; visiting /HomeServlet afterward greets the user by name. If accessed directly without logging in first (no active session), HomeServlet redirects to login.html instead.

Q4. MVC Refactor — Flipkart-style Product Listing

Layer	File	Responsibility
Model (data)	Product.java	Plain JavaBean: id, name, price.
Model (data access)	ProductDAO.java	All JDBC code lives here — the only class that talks to the database.
Controller	ProductController.java	Servlet: calls ProductDAO, sets request attributes, forwards to the View. Contains
View	productList.jsp	Reads request attributes and renders the HTML table. Contains no JDBC or busi

```
// ProductController.java - Controller: NO HTML anywhere in this class
@WebServlet("/products")
public class ProductController extends HttpServlet {

    private final ProductDAO productDAO = new ProductDAO();

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        try {
            List<Product> products = productDAO.getAllProducts();
            request.setAttribute("productList", products);
        } catch (SQLException e) {
            request.setAttribute("dbError", "Unable to load products right now.");
        }

        RequestDispatcher dispatcher = request.getRequestDispatcher("productList.jsp");
        dispatcher.forward(request, response);
    }
}
```

This separation means the DAO could be swapped for a different database with zero changes to the Controller or View, and the View's HTML could be redesigned without touching any Java/SQL code — each layer has exactly one responsibility.

Q5. Spring MVC vs Plain Servlets/JSP — Short Note

Spring MVC replaces the manual boilerplate of writing web.xml mappings and `RequestDispatcher.forward()` calls by hand with annotation-based routing (`@Controller`, `@RequestMapping/@GetMapping`) and automatic dependency injection, so classes like `ProductDAO` would be `@Autowired` into the controller instead of manually instantiated. Views are typically resolved and rendered through a template engine (like Thymeleaf) rather than raw JSP scriptlets, keeping the View layer cleaner and closer to plain HTML. Spring also bundles built-in support for form validation, exception handling, and RESTful JSON APIs, reducing the amount of repetitive plumbing code needed compared to hand-written Servlet/JSP MVC. Overall, the same Model-View-Controller separation is preserved, but Spring MVC handles much more of the 'wiring' automatically, making larger applications easier to structure and maintain.